Minimum Viable State: What Adam Must Carry Across a Preemption

Ritesh Roshan Sahoo

Abstract—A training job on a preemptible machine gets thirty seconds to two minutes of warning before it is taken away. In that time it must move its state, which for Adam is three times the size of the model. Elastic training systems move all of it. We ask which parts must cross. We interrupt a run, resume it from one of 15 reduced payloads, and compare it with a bit-identical uninterrupted run, over 540 migrations, three model sizes and 3 seeds, reading every gap against the seed-to-seed spread. Moments quantised to eight bits, half the bytes, cannot be told from the full state. Dropping the first moment and sending only the row and column sums of the second cuts the payload to a third, or a sixth with half-precision weights, and stays inside that spread. Dropping both moments does not, and the destination then decides the outcome: keeping the step counter is worst, re-estimating the second moment alone is best. A short model of the update predicts each transient. We time drain and restore, resume on a CPU and two GPUs, and give a rule for what to send within a notice window. The largest model has 25.3 million parameters.

Index Terms—Elastic training, spot instances, preemption, checkpointing, optimizer state, Adam, state migration, fault tolerance, cloud economics.

1 Introduction

The cheapest way to rent an accelerator is to accept that it can be taken back. A spot instance is reclaimed with a warning of two minutes on one large cloud and at most thirty seconds on two others [1], [2]. In that window a training job has to get its state off the machine. How much state is that? For a model trained with Adam [3], [4] it is the weights and two more tensors of the same shape, a running average of the gradient and a running average of its square: twelve bytes per parameter in single precision. A 6.74-billion-parameter model owns 80.9 GB of it. Over a 10 Gb/s link that takes 64.7 s to drain before a single byte has been serialised. It fits a two-minute warning. It does not fit a thirty-second one.

The systems that keep training alive on such machines differ in almost everything except this. They re-plan pipelines when workers vanish [5]–[8], replicate state to peers [9], rebuild it from mirrored gradients [10], migrate it directly between nodes [11], or snapshot the whole device [12], [13]. All of them move the weights and both moments in full. The most recent migration system says so outright: "it is necessary to migrate model parameters, optimizer states and other states" [11]. The schedulers that decide when to use spot capacity then fold everything that happens at a switch into one constant, the changeover delay [1].

Is it necessary? There is a reason to doubt it. The two moments are not like the weights. The weights are the result of the whole run. The moments are short memories: with the usual settings the first one remembers about ten steps and the second a hundred to a thousand. Work on optimizers has been throwing parts of them away for years. Adafactor trains with only the row and column sums of the second moment [14]; later work shares or removes most of it for the whole run [15], [16] and stores both moments in eight or four bits [17], [18]. If a run can live with a coarse second moment at every step, it can presumably survive being handed a coarse one once. Nobody has measured that. The question is not whether the state can be made smaller, which checkpoint compression already answers [19], [20]. It is which of it has to exist on the wire at all, and what is paid in training progress for each part left behind.

We answer by experiment. We stop a run at step T, reduce its state according to a *strategy*, and resume from what is left (Fig. 1). Then we compare the resumed run, step by step, with a control that was never interrupted. Two precautions make the comparison mean something. First, data order, dropout masks and the learning rate are all pure functions of the step index, so the control and the resumed run differ in exactly one thing, the state that crossed; a resume from the full state reproduces the control bit for bit. Second, two runs that differ only in their seed also drift apart, so we read every gap against that spread, which we call the *seed band*. A migrated run that stays inside it cannot be told from a different seed of the same job, and we do not count it as damaged.

Our contributions are these.

- **A controlled ablation of optimizer state at a migration boundary.** We evaluate 15 payload strategies, from the full state down to the weights alone, over 540 migrations on three model sizes, four points in training and 3 seeds, with bootstrap intervals over seeds and censored recovery times (Sections 5 and 6).
- **An account of the arrival transient.** With both moments zeroed, the first update is not small, as is sometimes assumed, but about $\sqrt{(1-\beta_1^2)/(1-\beta_2)}$ times the steady one, and it dies away on the horizon of the second moment if the step counter is kept and

• *R. R. Sahoo is with the School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Faridabad, Haryana, India.*
E-mail: riteshroshansahoo01@gmail.com.

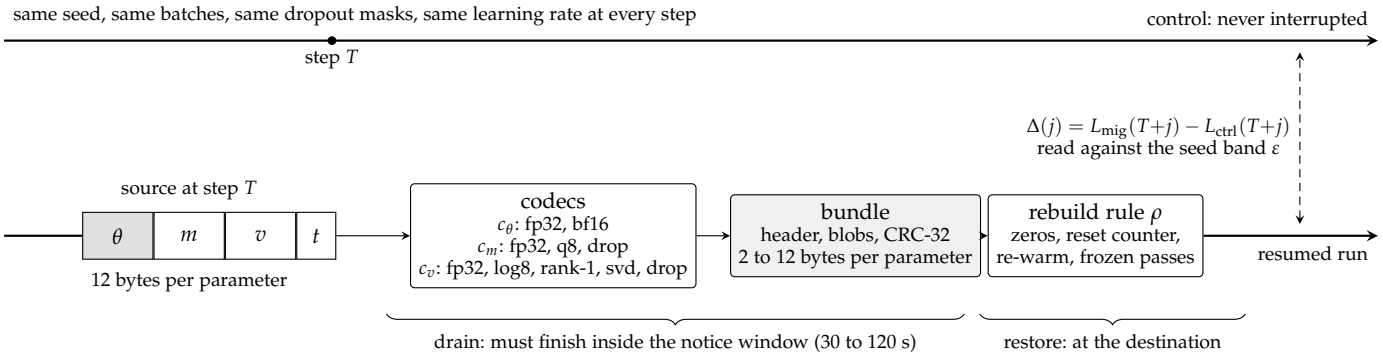


Fig. 1. The experiment. A run is stopped at step T; a strategy chooses a codec for the weights and for each moment, or drops the moment, and a rule for what the destination does about what is missing. The resumed run is compared at every later step with a control that shares its seed, data order and dropout masks, so the only difference between them is the state that crossed.

of the first if it is reset. The closed forms agree with measured update sizes. They explain why the first moment may be dropped, why the second may be sketched, why a kept counter hurts, and why a first moment re-estimated at the destination is harmful (Section 4).

- **Measured bytes and latency.** We take payload sizes from the serialised bundle, where they agree with the count from tensor shapes to within 0.04%, and we time drain and restore per stage over repeated rounds (Section 6.4).
- **Migration across device classes.** We resume the same payload on a CPU, a discrete GPU and an integrated GPU of a different vendor, and measure the gap between devices against the same seed band (Section 6.5).
- **A policy.** A cost model takes the measured curve, a notice window, a bandwidth and a price, and returns the cheapest strategy that fits (Section 7). Its outputs for models larger than those we trained are extrapolations and are labelled as such.

We should say at once what this is not. Everything ran on one laptop, with models of 0.8 to 25.3 million parameters, and we say so wherever it matters. What we offer at that scale is a method, a noise floor and a set of mechanisms that make predictions. Section 9 lists what remains to be checked on a real cluster.

2 Background and Related Work

2.1 Adam and its state

Adam [3] keeps, for every parameter, an exponential average of the gradient and of its square. With gradient g_t at optimizer step t,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, \tag{2}$$

$$\theta_t = \theta_{t-1} - \eta_t \, \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \epsilon}, \tag{3}$$

with decoupled weight decay in AdamW [4]. The state of a run is (θ, m, v), the step count t that enters the two bias corrections, and a few scalars. The first moment averages over a horizon of about $1/(1-\beta_1)$ steps, ten at the usual $\beta_1 = 0.9$; the second over $1/(1-\beta_2)$, a hundred to a thousand. The moments are updated *before* the step is taken, so a moment that arrives as zero is already partly refilled when it is first used. Section 4 depends on this ordering.

2.2 Training on preemptible and elastic resources

Varuna [5], Bamboo [6], Oobleck [7], Parcae [8], ReCycle [21], Lazarus [22] and TrainMover [23] keep a pipeline-parallel job running as workers disappear, by re-planning, redundant computation or replicated stages. Parcae keeps a CPU-side copy of "parameters and optimizer states" so that a preempted worker's state survives [8]. Elastic schedulers such as Pollux [24] and EasyScale [25] resize jobs on shared clusters; EasyScale requires the resized job to reproduce the fixed-size one bit for bit, the standard we adopt for our control. ETC [11] moves state directly between nodes without a checkpoint; Singularity [26], CRIUgpu [12] and PhoenixOS [13] snapshot the device transparently, which carries at least the resident sixteen bytes per parameter. On the buying side, SkyPilot [27], Can't Be Late [1] and SkyNomad [28] choose between spot and on-demand capacity and between regions; they model recovery as a fixed delay or a fixed migration cost. In every one of these systems the optimizer state is cargo. None varies what is carried, so none can say what a smaller cargo costs.

2.3 Checkpointing

CheckFreq [29] tunes how often to checkpoint; Gemini [9], Just-in-Time checkpointing [30], Swift [31], Checkmate [10] and DataStates-LLM [32] make saving or recovering faster while preserving the full state. A second line makes checkpoints smaller: Check-N-Run [33] and Inshrinkerator [34] store them lossily, ExCP [19] prunes weights and moments jointly, and MoC-System [35] and MoEvement [36] save only some experts of a mixture-of-experts model at each checkpoint, the nearest precedent for partial state. These papers report compression ratio against final accuracy after one or more lossy restores. Their concern is the size of what is stored; the transient that follows a restore, which decides whether a migration is worth making, is not resolved. Kokolis et al. [37] supply the reliability data and the effective-training-time ratio in which such costs are counted at scale.

2.4 Smaller optimizer state

Adafactor [14] replaces v for a matrix parameter by its row and column sums. Adam-mini [15] and SlimAdam [16] show that most per-coordinate second moments can be shared within blocks; SlimAdam finds that embedding matrices resist this along the vocabulary dimension, which is why we test a strategy that keeps embedding moments exact. Eight-bit and four-bit optimizers [17], [18] store quantised moments throughout training. All of these keep the reduced state for the whole run and tune for it. Our question is different. A run trained with exact Adam is handed a reduced state *once*, and has to carry on.

2.5 Warm-up, bias correction and restarts

A freshly initialised Adam takes oversized steps, which is what learning-rate warm-up compensates [38]–[40]; bias correction can be read as an implicit schedule [41]. Adaptive methods also run at the edge of stability, where the gradient oscillates across sharp directions [42]; Section 4 uses that fact. Resetting Adam on purpose helps in reinforcement learning, where the objective changes [43]; here the objective is stationary and nothing in the moments is stale.

2.6 Concurrent work

Two preprints are closest. Mitra [44] studies training under scheduled interruptions and reports that a weights-only checkpoint costs final perplexity and accuracy relative to a full one, and that rebuilding a decayed first moment recovers about a fifth of the penalty, not robustly across seeds. That study measures end-of-run quality; it does not separate the two moments, account for bytes or resolve the transient, and its restarts change data order. Zhang et al. [20] compress both moments under a bound on the first restored update and cut moment payloads by about 70%; they do not drop or rebuild a moment. We use their first-update criterion as a diagnostic (Section 6.3), and the quantised strategy below occupies the same point on the trade-off curve. Relative to both, we add the step-resolved measurement against a deterministic control and a noise floor, the separation of m from v at equal bytes, the analysis of why each transient has the shape it has, and the conversion of the result into a migration decision. Table 1 sets the closest work side by side.

Seed-to-seed variation is large enough to matter for any such comparison [45]–[47], and results are not guaranteed to reproduce across device types [48]–[50]; both facts shape the method that follows.

3 Payload Strategies

3.1 What a migration carries

A migration is a function from the state of the source to the state the destination starts with. It is described by a *strategy*, a tuple $(c_\theta, c_m, c_v, \rho)$: a codec for the weights, a codec for each moment or nothing if that moment is left behind, and a rule ρ that says what the destination does about what is missing. The step index, Adam's step counter, the model's non-trainable buffers and a fingerprint of the configuration always travel; together they are a few hundred bytes plus the buffers. Table 2 lists the strategies we study.

Codecs. `fp32` sends a tensor as it is and `bf16` rounds it to sixteen bits. `q8` is a block-wise linear quantiser: each block of 256 values is scaled by its largest magnitude and stored as signed bytes with one scale per block. `log8` is its counterpart for the non-negative second moment, which spans many orders of magnitude within one tensor: the logarithm of each positive entry is quantised to 254 levels between the block's smallest and largest, and the code zero is kept for exact zeros so that a coordinate that never received gradient stays at zero. `rank1` sends, for a tensor reshaped to a matrix $V \in \mathbb{R}^{p\times q}$, only its row sums r and column sums c, and the destination rebuilds $\hat{V} = rc^\top / \sum_i r_i$. This is the Adafactor estimator [14], applied once at the boundary; it is non-negative by construction, so no entry of the rebuilt second moment can reach the square root with the wrong sign. Vectors are sent whole. `rank1e` does the same but sends embedding and output-projection moments exactly, since their rows receive gradient only when their token is sampled. `svdk` sends a rank-k factorisation of $\sqrt{V}$ and squares the product at the destination, which is again non-negative.

Reconstruction. The rule ρ takes one of six values. `none` applies when both moments arrive. `zeros` sets a missing moment to zero and keeps the step counter. `fresh` zeroes both moments and resets the counter, which is what constructing a new optimizer does. `rewarm` does the same and also ramps the learning rate linearly from zero to its scheduled value over 100 steps. `warm` and `warmv` spend 20 forward-backward passes at the destination, on batches drawn from a stream that training never uses, with the weights held fixed, and install the bias-corrected averages of g and g^2 as moments; `warm` rebuilds whichever moment is missing and `warmv` rebuilds only v and leaves m at zero. These passes cost compute at the destination and no bytes.

The strategies fall into four families: both moments carried (exactly, quantised, or with v sketched), m dropped, v dropped, and both dropped. `w_v` and `w_m` cost the same bytes and differ only in which moment they keep; `w_zero` and `w_fresh` carry the same bytes and differ only in the counter.

3.2 Wire format and byte accounting

A payload is one self-describing bundle: a short magic string and version, a JSON header that names every tensor with its codec, shape, offset and CRC-32, and the tensor blobs aligned to 64 bytes. The header carries the strategy, the step, the Adam counter and the configuration fingerprint, and the destination reports a bundle whose fingerprint differs from its own configuration. Files are written to a temporary name, flushed to stable storage and renamed, so that a process killed while draining leaves the previous payload or the complete new one. A single module owns serialisation, and every size we report is the length of the bundle it produced. Table 2 compares those lengths with the count obtained from tensor shapes alone; the largest disagreement is 0.04%, which is the header and alignment.

Moving a payload is timed in nine stages: encode, device-to-host copy, serialise and write at the source; read, deserialise, host-to-device copy, decode and rebuild at the destination. The first four are the *drain*, which must finish inside the notice window; the last five are the *restore*.

TABLE 1
The work this paper builds on, by what state each moves when a job is interrupted and what each measures.

Work	Purpose	State that moves	What is measured
Parcae [8]	spot training, proactive re-planning	full; CPU-side copy of parameters and optimizer states	throughput and cost on a replayed spot trace
Can't Be Late [1]	spot or on-demand under a deadline	not modelled; one constant changeover delay	cost saving on availability traces
EasyScale [25]	elastic resizing	full	bitwise equality with the fixed-size run
ETC [11]	migration without a checkpoint	full, peer to peer	migration overhead
Checkmate [10]	checkpointing by gradient replication	full, replayed on a shadow cluster	repeated work per failure, throughput
CRIUgpu [12]	transparent device snapshot	all device memory	checkpoint and restore time
ExCP [19]	checkpoint compression	pruned four-bit weights and moments	final accuracy after restores
MoEvement [36]	sparse checkpoints for mixtures of experts	a subset of experts per checkpoint	checkpoint and recovery overhead
Zhang et al. [20]	guarded moment compression	weights and log-coded moments	first-update error, loss after 100 updates
Mitra [44]	training under scheduled interruptions	weights, or weights and a decayed m	final perplexity and accuracy
This paper	ablation at the migration boundary	15 payloads, from full to weights alone	held-out loss at every step against a bit-identical control and a seed band; measured bytes and latency

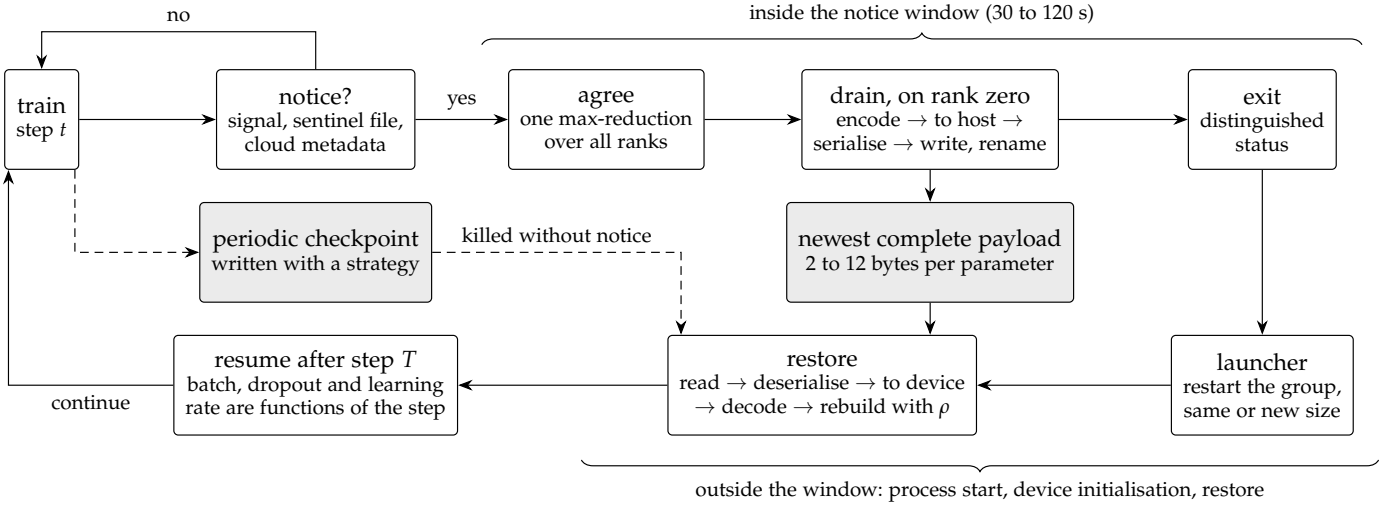


Fig. 2. The runtime. Every rank checks for a preemption notice at the top of each step, and one max-reduction makes all ranks stop at the same step. Rank zero drains the configured payload and the group exits; a launcher restarts it, at the same or a different size, from the newest complete payload. A failure that arrives without notice falls back to the last periodic checkpoint (dashed). Because data order is a function of the step, the restarted group needs nothing but the bundle.

3.3 Runtime

The runtime that uses these payloads is small (Fig. 2). Each rank watches for a preemption notice from any of four sources: a termination signal, a sentinel file, and the instance-metadata endpoints of two clouds. At the top of every step the ranks agree on whether anyone has been notified with one max-reduction, so that all of them stop at the same step. Rank zero then drains the configured strategy and the group exits with a distinguished status. A launcher restarts the group, at the same or a different size, from the newest complete payload; a periodic checkpoint, itself written with one of the strategies, covers failures that arrive without notice. Collectives use the Gloo backend so that the same code runs on CPUs and on a single GPU.

Because batches are a function of the global step rather than of loader state (Section 5.1), a resized group sees the same data as the original, and nothing about the data pipeline has to be checkpointed.

4 The Arrival Transient

Before looking at any data we can work out what should happen in the first steps after a reduced state arrives. It takes nothing more than (1)–(3), and the answer tells us which strategies are worth running and what to expect from each. Number the steps after arrival $k = 1, 2, \ldots$ and consider one coordinate whose gradient over this short window has mean μ and variance σ^2, with independent noise from step to step. Let $R(k)$ be the root-mean-square size of the update at step k divided by its size in the uninterrupted run. Two limits bracket the behaviour: a coordinate dominated

TABLE 2
Strategies and their payloads in bytes per parameter at 25.3M parameters. Analytic sizes are computed from tensor shapes; measured sizes are the serialised bundle, including header, padding and checksums.

Strategy	c_θ	c_m	c_v	ρ	Analytic	Measured	Δ %
`full`	fp32	fp32	fp32	none	12.000	12.001	+0.01
`w_mq8_vlog8`	fp32	q8	log8	none	6.047	6.049	+0.03
`w_m_vr1`	fp32	fp32	rank1	none	8.012	8.013	+0.01
`w_v`	fp32	–	fp32	zeros	8.000	8.001	+0.01
`w_vsvd4`	fp32	–	svd4	zeros	4.044	4.044	+0.02
`w_emb_v`	fp32	–	rank1e	zeros	4.038	4.038	+0.02
`w_vr1`	fp32	–	rank1	zeros	4.012	4.013	+0.02
`wbf16_vr1`	bf16	–	rank1	zeros	2.012	2.013	+0.04
`w_m`	fp32	fp32	–	zeros	8.000	8.001	+0.01
`w_zero`	fp32	–	–	zeros	4.000	4.000	+0.01
`w_fresh`	fp32	–	–	fresh	4.000	4.000	+0.01
`w_rewarm`	fp32	–	–	rewarm	4.000	4.000	+0.01
`w_warmv`	fp32	–	–	warmv	4.000	4.000	+0.01
`w_m_warm`	fp32	fp32	–	warm	8.000	8.001	+0.01
`w_warm`	fp32	–	–	warm	4.000	4.000	+0.01

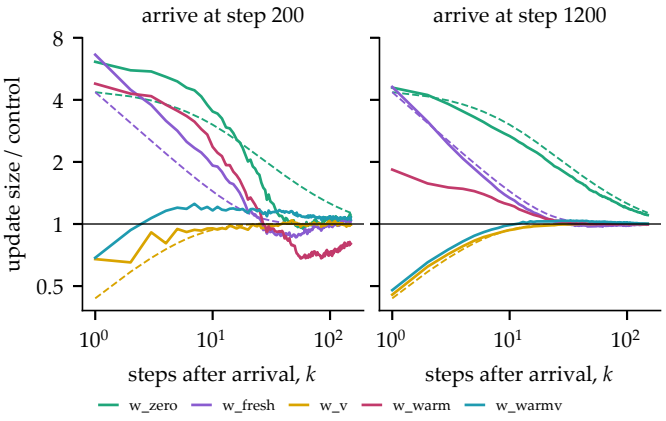


Fig. 3. Size of each update after arrival, divided by the size of the same update in the uninterrupted run (gpt-s, mean of 2 seeds). Dashed lines are the noise-dominated column of Proposition 1, with no fitted parameter.

by noise ($\mu = 0$) and one dominated by signal ($\sigma = 0$). In the uninterrupted run the noise-dominated update has size $\eta\sqrt{(1-\beta_1)/(1+\beta_1)}$, about $0.23\,\eta$ at $\beta_1 = 0.9$, and the signal-dominated one has size η. The Adam counter at arrival is large, so that both bias corrections equal one unless the counter is reset.

Proposition 1 (Size of the arrival transient)**.** *Approximating v by its expectation, the update inflation $R(k)$ is as follows.*

State on arrival	*Noise-dominated*	*Signal-dominated*
$m = v = 0$, *counter kept*	$\sqrt{\dfrac{1-\beta_1^{2k}}{1-\beta_2^{k}}}$	$\dfrac{1-\beta_1^{k}}{\sqrt{1-\beta_2^{k}}}$
$m = v = 0$, *counter reset*	$\sqrt{\dfrac{1+\beta_1^{k}}{1-\beta_1^{k}}}$	1
$m = 0$, v *carried*	$\sqrt{1-\beta_1^{2k}}$	$1-\beta_1^{k}$
m *carried*, $v = 0$	$1/\sqrt{1-\beta_2^{k}}$	$1/\sqrt{1-\beta_2^{k}}$

Proof: From (1) with $m_0 = 0$, $m_k = (1-\beta_1)\sum_{i\le k}\beta_1^{k-i}g_i$, so $\mathbb{E}\,m_k = (1-\beta_1^k)\mu$ and $\mathrm{Var}\,m_k = \frac{1-\beta_1}{1+\beta_1}(1-\beta_1^{2k})\sigma^2$; likewise $\mathbb{E}\,v_k = (1-\beta_2^k)(\mu^2+\sigma^2)$ from (2). A carried moment keeps its stationary value. Rows one, three and four are ratios of these to the stationary values. Resetting the counter divides m_k by $1-\beta_1^k$ and v_k by $1-\beta_2^k$, which gives row two. At $k = 1$ the first row is exact, with no approximation: the update is $\eta\,(1-\beta_1)/\sqrt{1-\beta_2}$ times the sign of the gradient on every coordinate. □

Four consequences follow, and the experiments test each of them.

The first update after a cold arrival is large, not small. With $\beta_1 = 0.9$ and $\beta_2 = 0.99$ a noise-dominated coordinate moves 4.4 times further than it would have; with $\beta_2 = 0.999$, 13.8 times. The intuition that zeroed moments give a near-zero first step holds only for optimizers that apply the step before updating the moments.

Keeping the step counter makes a cold arrival worse. With the counter kept, the inflation in row one decays as $1/\sqrt{1-\beta_2^k}$, on the horizon of the *second* moment: the excess movement, $\sum_k (R(k)-1)$, grows in proportion to $1/(1-\beta_2)$ and is 14 steps' worth at $\beta_2 = 0.95$, 109 at 0.99 and 1292 at 0.999. With the counter reset, the bias correction supplies exactly the factor that v lacks, the inflation decays on the horizon of the *first* moment, and the excess is 16 steps' worth at any β_2. In the signal-dominated limit the kept counter overshoots by up to 2.13 at step 13 (6.57 for $\beta_2 = 0.999$) where the reset counter does not overshoot at all. A reset is the right companion to an empty optimizer, and the wrong one to a carried moment.

The first moment is cheap to lose. With v intact and m refilling from zero the update is too *small*, by $\sqrt{1-\beta_1^{2k}}$, and only for about $1/(1-\beta_1)$ steps. The shortfall sums to 2.5 steps of progress. Nothing can diverge. This is why every strategy in the middle family drops m.

The second moment must never be the only thing missing.

Corollary 1. *If m is carried and v is zeroed with the counter kept, the first update is at least $1/\sqrt{1-\beta_2}$ times the steady one in root-mean-square, and its norm is dominated by coordinates whose current gradient happens to be small.*

The carried m is divided by $\sqrt{(1-\beta_2)}\,|g_1|$, and m is independent of g_1, so a coordinate with a small $|g_1|$ takes an arbitrarily large step. When both moments are zeroed this cannot happen, because m_1 is then proportional to the same g_1. We run `w_m` as the measured counterpart of this statement; the strategy validator otherwise rejects it.

The proposition also says what the sketch of v has to preserve. Only $\sqrt{v}$ enters the step, coordinate by coordinate; a sketch that gets its scale right within a tensor moves the update by the relative error of $\sqrt{v}$, not by an order of magnitude, and the error is forgotten on the horizon $1/(1-\beta_2)$ as true squared gradients arrive.

Measured update sizes. Fig. 3 overlays the prediction on the measured size of every update for the first 150 steps after arrival. At 60% of the schedule the closed forms follow the measurement from the first step to the last. With both moments zeroed and the counter kept, the first update is 4.59 times the control's (predicted 4.36), the tenth 2.67 (3.03)

and the hundredth still 1.20 (1.26). With the counter reset the first update is the same size, 4.62, and the tenth is down to 1.34 (1.44); by the hundredth it is 0.99. With only m dropped the first update is 0.45 of the control's (0.44) and the tenth 0.94 (0.94); the rank-one sketch of v gives 0.46 and 0.94, indistinguishable from the exact v, and the quantised moments give 1.00 throughout. At 10% of the schedule, where the learning rate is at its peak and the loss is still falling fast, the gradient is not pure noise and the weights move during the transient; the initial inflation is larger than the noise-dominated value (6.12 and 6.62) and the run with the kept counter falls back to the control's step size sooner than predicted, after the damage of Section 6.2 is done. The ordering of the strategies is the same at both points.

A first moment cannot be re-estimated at a point. The `warm` rule rebuilds m as the mean of 20 gradients at the arrival weights. Under the model above this would have nearly the same size as the carried m, since averaging 20 independent gradients removes about as much noise as the exponential average does. It does not: the rebuilt m is 4.0 times larger than the one left behind (Section 6.3). The model's assumption of independent gradients is what fails. Along a training trajectory successive gradients are anti-correlated in the directions of high curvature, where adaptive methods oscillate [42], and the exponential average cancels them. Gradients taken repeatedly at one point all contain that component with the same sign. The moment m is therefore a statistic of the trajectory and not of the point where it stopped, and an estimate made at a standstill overstates it. Squared gradients do not cancel, so v can be re-estimated at a point; that is the `warmv` rule.

5 Experimental Method

5.1 One variable

A migrated run and its control must differ only in the state that crossed. Three mechanisms enforce this. The batch used at step t is drawn from a generator seeded by a hash of the run seed, the stream and t, so the data order is a function of the step index and no loader state exists to be restored incorrectly. Dropout masks are seeded the same way. The learning rate is a function of t. Training, the destination's warm-up passes and held-out evaluation use three separate streams, so the warm-up passes neither consume nor repeat training batches. Deterministic kernels are enforced in the framework. Under these conditions a migration with the `full` strategy reproduces the control's loss at every later step bit for bit, on the same device; the test suite asserts this, and `full` appears in the tables as a row of exact zeros. This is the bitwise standard EasyScale sets for elastic training [25], used here as an instrument: any gap in any other row was caused by the missing state.

Each control saves a full snapshot at every migration point. A migration cell loads the snapshot into a source process, packs the strategy's payload, writes it, reads it into a freshly built destination, rebuilds what is missing and trains for $W = 300$ further steps. Nothing but the bundle passes from one to the other.

5.2 Metrics

Loss is evaluated every 10 steps on a fixed held-out set of 512 sequences. Training loss is too noisy at these sizes and is confounded with which batch lands on which step. Let L_{mig} and L_{ctrl} be the held-out losses of a migrated run and of its own control, with the same seed, and $\Delta(j) = L_{\text{mig}}(T + j) - L_{\text{ctrl}}(T + j)$.

Seed band. For each model size and migration point, ε is the 95th percentile of $|L_a(t) - L_b(t)|$ over all pairs of control seeds $a \neq b$ and all evaluations in $(T, T + W]$. It measures how far apart two runs of the same job are for no reason at all. A migrated run with $\Delta \leq \varepsilon$ is no further from its control than another seed would be. With 3 seeds there are three such pairs at each point, so the band is a coarse estimate; across the sweep it lies between 10.0 and 36.2 mnats.

Peak excess loss is $\max_j \Delta(j)$, reported in thousandths of a nat (mnats) and as a multiple of ε. It is the main quantity in the tables because it is defined for every run and has a direct reading: a ratio below one means the run never left the band.

Steps to recovery (StR) is the first offset k such that $\Delta(j) \leq \varepsilon$ for all $j \in [k, k + 50]$. One lucky crossing does not count. A run with no such k inside the window is censored and reported as $> W$; it is never assigned the window length. Tables give the number of runs with StR $= 0$, which never left the band, the median StR of those that left and returned, and the number censored.

Regret is $\sum_j \max(0, \Delta(j))$ over the window, in nat-steps. It separates a deep, brief transient from a shallow, long one.

Steps lost converts the training-loss gap at the end of the window into the number of steps by which the migrated run trails its control, read off the control's smoothed loss curve, plus any warm-up passes spent. It is the unit a scheduler wants, and it is noisy: applied to pairs of controls it gives values of 4 to 216 steps with no migration at all. We report it with that caveat and use it only in the cost model.

Uncertainty. The migration points of one seed share a control and are not independent. Intervals are therefore 95% percentile intervals from 10000 bootstrap resamples of whole seeds.

5.3 Workloads and hardware

The main workload is a decoder-only transformer trained on character-level text [51] at three sizes, 0.8M, 10.7M and 25.3M parameters, for 2000 steps with AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay 0.1, gradient clipping at 1, a 100-step warm-up and cosine decay, batch 64 and context 256, under bfloat16 autocast with single-precision master weights and moments. Each size is run with 3 seeds and migrated at steps 200, 600, 1200, 1700, that is, 10%, 30%, 60% and 85% of the schedule.

Everything ran on one laptop: an Intel Core Ultra 9 185H with 32 GB of memory, an NVIDIA RTX 4050 Laptop GPU with 6 GB, and the processor's integrated Arc GPU, under Windows 11 with PyTorch 2.14 (CUDA 12.6) and, for the integrated GPU, PyTorch 2.15. Every run writes a manifest with the commit, the resolved configuration, the device, the measured payload size and the median step time.

TABLE 3
Cost of each strategy on the 25.3M-parameter model, over all four migration points and 3 seeds. Payload is the share of the full state's bytes. Peak is the largest excess held-out loss in the 300 steps after arrival; Peak/ε divides it by the seed band of the same cell, so values below one are inside the noise floor. Brackets are 95% seed-bootstrap intervals. StR $= 0$ counts runs that never left the band; StR is the median recovery time, in steps, of those that left and returned; $> W$ counts runs still outside at the end of the window.

Strategy	Payload %	Peak (mnats)	Peak/ε	Regret (nat-steps)	Steps lost	StR $= 0$	StR	$> W$
`full`	100.0	0.0 [0.0, 0.0]	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]	0.0 [0.0, 0.0]	12/12	–	0
`w_mq8_vlog8`	50.4	2.2 [1.9, 2.6]	0.09 [0.08, 0.11]	0.12 [0.10, 0.14]	1.1 [0.5, 1.9]	12/12	–	0
`w_m_vr1`	66.8	4.1 [3.3, 5.2]	0.17 [0.14, 0.22]	0.23 [0.16, 0.31]	4.4 [3.2, 6.7]	12/12	–	0
`w_v`	66.7	6.8 [6.7, 7.1]	0.29 [0.28, 0.30]	0.72 [0.60, 0.79]	8.2 [6.5, 11.2]	12/12	–	0
`w_vsvd4`	33.7	7.9 [7.1, 8.7]	0.33 [0.30, 0.36]	0.80 [0.68, 0.87]	8.7 [7.2, 11.5]	12/12	–	0
`w_emb_v`	33.7	7.4 [6.8, 8.4]	0.31 [0.29, 0.35]	0.82 [0.78, 0.89]	8.9 [7.3, 11.7]	12/12	–	0
`w_vr1`	33.4	7.4 [7.1, 8.1]	0.31 [0.30, 0.34]	0.80 [0.79, 0.80]	9.1 [7.5, 12.1]	12/12	–	0
`wbf16_vr1`	16.8	7.5 [7.0, 7.9]	0.31 [0.29, 0.33]	0.85 [0.76, 0.91]	9.1 [7.6, 11.9]	12/12	–	0
`w_m`	66.7	1469.1 [1425.9, 1529.5]	69.26 [66.83, 72.29]	177.36 [167.40, 187.09]	634.6 [626.5, 646.2]	0/12	40	9
`w_zero`	33.3	271.3 [245.6, 288.5]	11.40 [10.30, 12.13]	19.80 [17.30, 22.41]	56.8 [51.7, 61.7]	3/12	75	3
`w_fresh`	33.3	105.7 [85.8, 139.9]	4.38 [3.57, 5.76]	6.40 [5.49, 8.07]	26.4 [23.5, 28.0]	5/12	25	3
`w_rewarm`	33.3	19.8 [18.9, 20.4]	0.83 [0.79, 0.85]	2.88 [2.75, 2.98]	35.3 [30.9, 41.5]	11/12	–	1
`w_warmv`	33.3	9.5 [8.4, 11.0]	0.40 [0.36, 0.46]	0.68 [0.58, 0.74]	27.2 [25.5, 29.9]	10/12	55	0
`w_m_warm`	66.7	13.1 [10.7, 17.0]	0.55 [0.45, 0.71]	0.40 [0.30, 0.59]	23.7 [22.3, 26.3]	9/12	20	0
`w_warm`	33.3	204.0 [187.8, 232.3]	8.44 [7.76, 9.63]	15.15 [13.44, 16.57]	64.2 [61.3, 68.1]	4/12	30	3

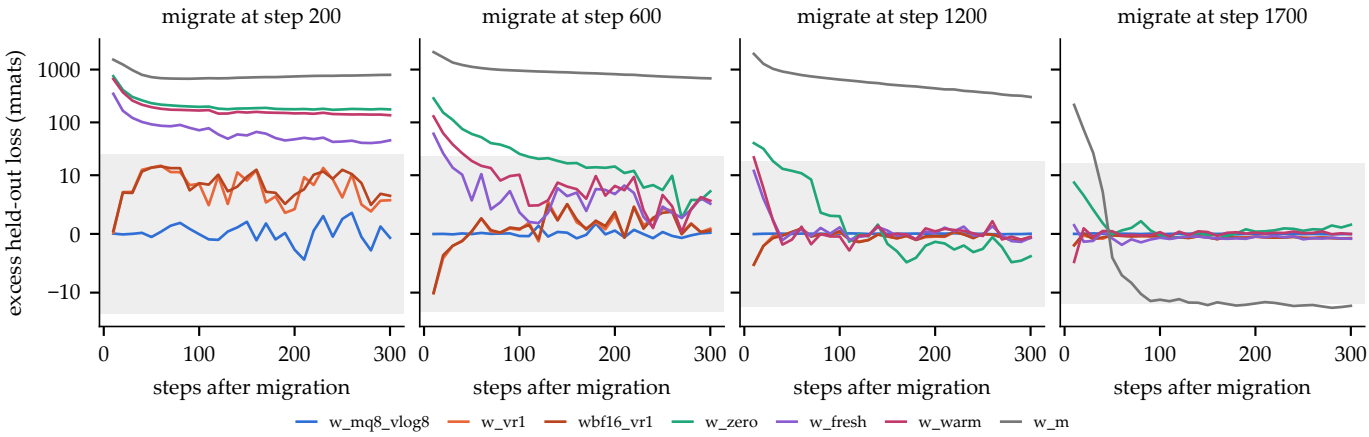


Fig. 4. Excess held-out loss after a migration at four points in training (25.3M parameters, mean over seeds; the vertical axis is linear within ± 10 mnats and logarithmic outside). The grey band is $\pm\varepsilon$, the spread between seeds of the uninterrupted run.

6 Results

6.1 What must cross

Table 3 is the whole experiment in one place, and Fig. 4 shows what the numbers look like as curves. The first row is the check on the instrument: the full state gives a gap of exactly zero at every step of every run. Everything below it is the price of leaving something behind. The rows fall into three groups, and the groups are far apart.

Coarse moments cost nothing. Quantising both moments to eight bits halves the payload, and the largest gap it ever opens is 0.09 of the seed band. All 12/12 runs stayed inside the band from the first evaluation to the last. Carrying the first moment exactly and sketching only the second, `w_m_vr1`, peaks at 0.17 with every run inside. If you ran the same job twice with different seeds you would see a difference ten times larger than this.

The first moment can stay behind. Take `w_v`: the weights and the exact second moment cross, the first moment is left behind and starts again from zero. Its peak is 0.29 of the band and no run leaves it. Now replace the exact second moment by its row and column sums, `w_vr1`. The payload falls from two thirds of the full state to a third, and the peak is 0.31: the sketch cannot be told from the exact tensor. Round the weights to sixteen bits as well, `wbf16_vr1`, and the payload is a sixth of the full state at a peak of 0.31. The two refinements we tried buy nothing. A rank-four factorisation of $\sqrt{v}$ and a sketch that keeps the embedding moments exact both land on the same value as the plain rank-one sketch. We had expected the embeddings to matter, because others have found that their second moments resist compression [16]. Here they do not, but our vocabulary is a few dozen characters and the embedding is a small tensor, so this says little about a model with fifty thousand tokens.

The second moment must not be the only thing missing. `w_m` costs exactly the bytes of `w_v` and keeps the other moment. It is the worst strategy in the table by a wide margin: a peak of 69.26 bands, no run that stays inside, and 9 of twelve still outside when the window closes. The two moments are not interchangeable, and Corollary 1 says why. A carried first moment divided by an empty second one is a step of the wrong size in every coordinate. The fault lies in the empty second moment and not in the carried first: rebuild v with 20 passes at the destination and keep the

carried m, `w_m_warm`, and the peak falls to 0.55 of the band.

If only the weights cross, the destination's choice decides the outcome. Five strategies send the same bytes, the weights alone, and differ only in what the destination does on arrival. They span a factor of nearly thirty. Zeroing the moments and keeping the step counter, `w_zero`, peaks at 11.40 bands. Building a new optimizer, which also resets the counter, `w_fresh`, peaks at 4.38. That is the order Proposition 1 predicts: the reset makes the bias correction supply the factor that the empty second moment lacks. Spending 20 passes to re-estimate both moments, `w_warm`, is worse than doing nothing, at 8.44, because the re-estimated first moment is too large (Section 6.3). The two options that work are the two that tame the first steps. Resetting the counter and ramping the learning rate again, `w_rewarm`, peaks at 0.83 and keeps 11/12 runs inside the band; it costs no computation, but the ramp itself gives up progress, and its regret is the largest of the strategies that stay inside. Re-estimating the second moment only and leaving the first at zero, `w_warmv`, peaks at 0.40 with 10/12 inside, and costs 20 forward-backward passes at the destination. So there is an answer for a job that can send nothing but its weights: rebuild v, do not rebuild m.

The picture is the same at every size. Fig. 5 plots bytes against damage for the three models, which span a factor of thirty in parameter count. The strategies that drop only the first moment sit at 0.22, 0.31 and 0.31 of the band from the smallest model to the largest. The damage from dropping both grows with size, from 2.51 to 9.58 to 11.40 bands for `w_zero`, so the separation between the groups widens as the model grows. The per-size table is in the supplement.

One column of Table 3 needs a warning. Steps lost is the unit a scheduler wants, and we use it in the cost model, but it is read off training loss and is noisy: two controls compared with each other show 4 to 216 steps with no migration at all. The nine steps lost by the rank-one strategies are inside that noise. Our conclusions rest on the held-out measures.

6.2 When it matters

The averages in Table 3 hide the strongest effect in the data, which is when the migration happens. Fig. 6 shows every cell of the sweep. Read any row from left to right within one model and the colour cools. From the first migration point to the last the largest model's seed band narrows only from 24.8 to 16.4 mnats; the damage falls by orders of magnitude. `w_zero` peaks at 750.1 mnats when the migration comes at 10% of the schedule and at 9.3 mnats when it comes at 85%, which is inside the band. `w_fresh` goes from 347.9 to 2.0, and `w_warm` from 663.1 to 1.2. Late in training almost anything works. Only `w_m` is still far outside, at 13.05 bands.

The reason is not mysterious. The transient is an inflated step, and the damage an inflated step does is proportional to the learning rate it is multiplied by and to how fast the loss is moving. At step 200 the learning rate has just reached its peak and the loss is falling steeply; at step 1700 the cosine schedule has brought the rate down about sixfold and the loss is nearly flat.

The same view shows where the cheap strategies come closest to the edge. At the earliest point the rank-one sketch peaks at 0.87 of the band on the largest model, and `w_warmv` at 1.15, just outside. A job that is preempted in its first tenth and wants a margin should send the quantised moments if they fit, which stay at 0.25 even there. The table of peaks by migration point, with intervals, is in the supplement.

TABLE 4
The first update after arrival against the true one (25.3M, mean over migration points). cos: cosine with the control's update. Size: norm ratio. $|m|$: norm of the installed first moment over the true one. $\sqrt{v}$ err: median relative error of the installed $\sqrt{v}$, in percent. The last four columns split cosine and error into matrices (mat.) and embedding tensors (emb.).

					cos		$\sqrt{v}$ err	
Strategy	cos	Size	$\lvert m\rvert$	$\sqrt{v}$ err	mat.	emb.	mat.	emb.
`full`	1.00	1.00	1.00	0.0	1.00	1.00	0.0	0.0
`w_mq8_vlog8`	1.00	1.00	1.00	0.1	1.00	1.00	0.1	0.1
`w_m_vr1`	0.99	1.01	1.00	7.9	0.99	0.99	7.9	8.3
`w_v`	0.45	0.50	0.00	0.0	0.45	0.43	0.0	0.0
`w_vsvd4`	0.45	0.50	0.00	5.7	0.45	0.43	5.7	6.7
`w_emb_v`	0.44	0.50	0.00	7.8	0.44	0.43	7.9	0.0
`w_vr1`	0.44	0.50	0.00	7.9	0.44	0.43	7.9	8.3
`wbf16_vr1`	0.44	0.50	0.00	7.9	0.44	0.43	7.9	8.3
`w_m`	0.22	175.86	1.00	100.0	0.23	0.12	100.0	100.0
`w_zero`	0.35	5.02	0.00	100.0	0.35	0.34	100.0	100.0
`w_fresh`	0.35	5.20	0.00	100.0	0.35	0.34	100.0	100.0
`w_rewarm`	0.35	0.05	0.00	100.0	0.35	0.34	100.0	100.0
`w_warmv`	0.43	0.54	0.00	22.6	0.43	0.42	22.6	16.2
`w_m_warm`	0.95	1.16	1.00	22.6	0.95	0.80	22.6	16.2
`w_warm`	0.22	2.60	3.99	22.6	0.22	0.18	22.6	16.2

6.3 First-update diagnostics

Why do the groups separate so cleanly? We can look directly at the first step each strategy takes and compare it with the step the control takes from the same weights on the same batch (Table 4). This is the diagnostic Zhang et al. use as an acceptance test for compressed moments [20]; we use it to explain the loss curves.

With both moments carried the first step is the true step, to two decimal places in direction and size, whether the moments are exact or quantised. With the first moment dropped, the step is half the true size and has a cosine of 0.45 with it. That sounds bad and is harmless: the step points along one fresh gradient instead of along the average of the last ten, and it is too short, never too long. The rank-one sketch gives the same step as the exact second moment, 0.44 and 0.5, because the sketch gets $\sqrt{v}$ right to within 8% in the median coordinate, on matrices and embeddings alike.

With both moments zeroed the first step is 5.0 times too long, close to the 4.4 that Proposition 1 gives for a coordinate dominated by noise, and the same whether the counter is kept or reset; the two differ only in what happens next (Fig. 3). With the first moment kept and the second zeroed it is 175.9 times too long. And `w_warm` shows the failure described at the end of Section 4: its second moment is off by 23%, which would be tolerable, but its first moment has 4.0 times the norm of the one it replaces, and the step it produces has a cosine of only 0.22 with the true one. The same rebuilt v with m left at zero, `w_warmv`, takes the same short, safe step as the other strategies that drop m.

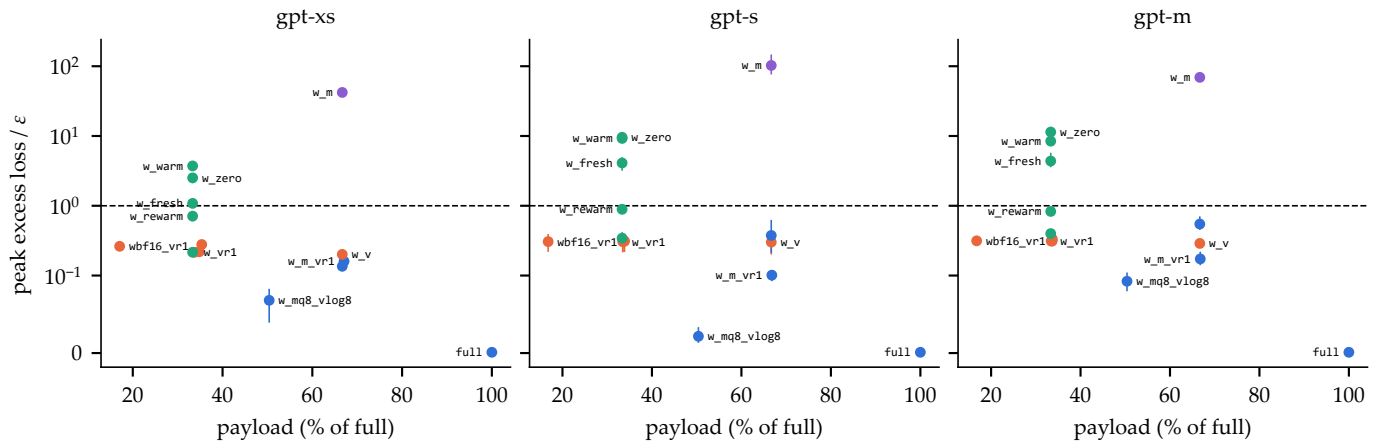


Fig. 5. Bytes against damage at the three model sizes. Each point is a strategy; the horizontal axis is its payload as a share of the full state, the vertical axis its peak excess loss in units of the seed band, with 95% intervals. The dashed line is the noise floor. Strategies that coincide with `w_vr1` are not labelled.

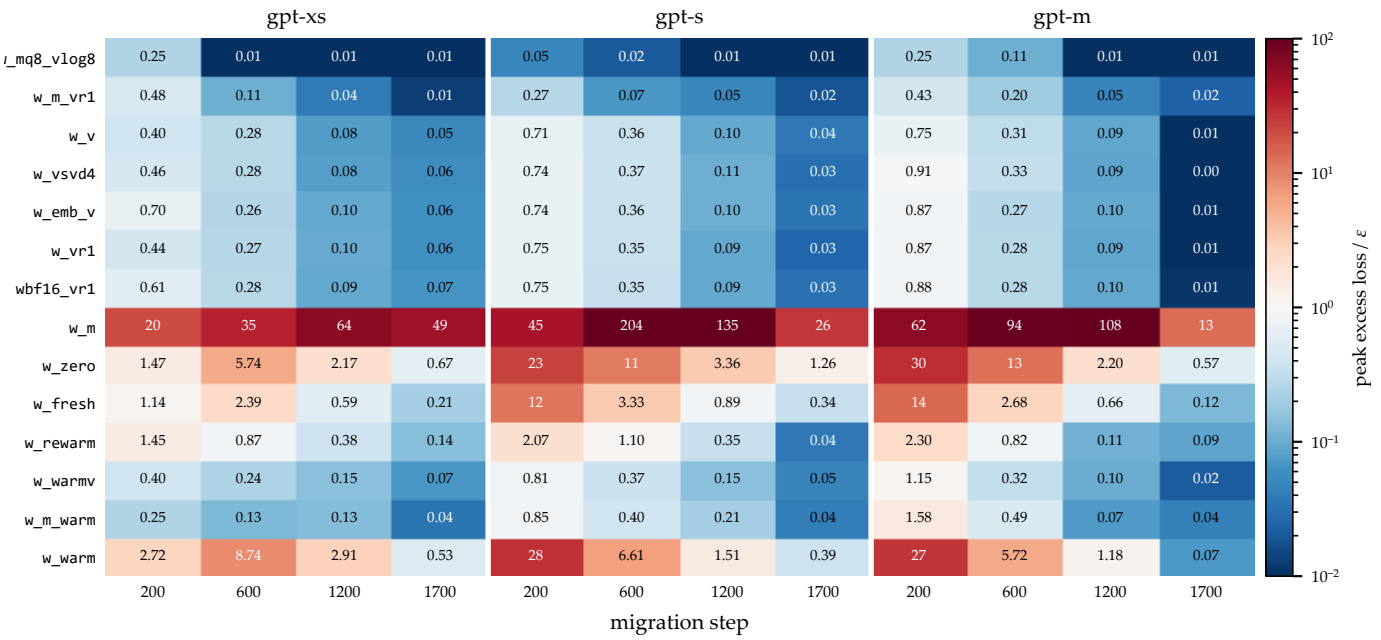


Fig. 6. Every cell of the sweep: peak excess held-out loss in units of the seed band, by strategy, model size and migration step (mean over seeds). Blue is inside the noise floor and red is outside.

6.4 Drain and restore latency

A smaller payload is only useful if making it does not cost more time than it saves. Fig. 7 breaks one migration into its stages; the full table is in the supplement. On this machine the full state, 304 MB, drains in 580 ms and restores in 427 ms, and nearly all of that is serialising, writing and reading bytes. The reduced payloads are faster in proportion to their size, because their encoders are cheap: the row and column sums take 3 ms to compute, and `w_vr1` drains in 219 ms and `wbf16_vr1` in 127 ms. The eight-bit quantisers cost 90 ms and still leave `w_mq8_vlog8` ahead of the full state at 442 ms.

Two entries go the other way. The rank-four factorisation spends 839 ms in the decomposition, so `w_vsvd4` drains in 1052 ms, slower than sending everything, for no gain in fidelity over the rank-one sketch. And the warm-up rules move their cost to the destination: 20 forward-backward passes take 5742 ms here, about twenty training steps, against 157 ms for the sketch. That is the price of `w_warmv`, and it is paid outside the notice window, where time is cheap.

These are times for local storage on one machine. They show that encoding is not the bottleneck; they say nothing about a network, which Section 7 models separately.

6.5 Across device classes

A preempted job rarely lands on the machine it left. Does the destination's hardware add a gap of its own? A payload is bytes, and any device that can run the model can read it, but two devices do not add floating-point numbers in the same order, and training amplifies small differences. So we cannot ask for bit-identical continuation across devices.

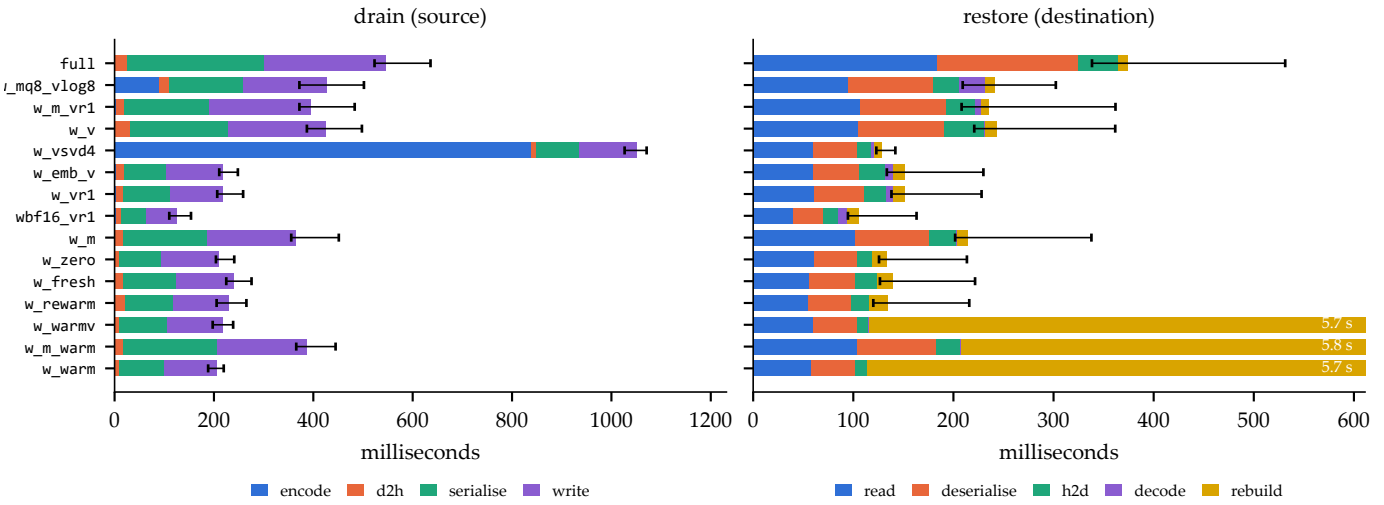


Fig. 7. Latency of one migration on the discrete GPU by stage (25.3M parameters, median of 20 rounds after warm-up rounds; whiskers are the 5th and 95th percentiles of the total). Drain is everything the source does inside the notice window; restore is everything the destination does before its first step. Bars that run off the right edge are the 20 warm-up passes, with their total written in; d2h and h2d are the copies from device to host and back.

TABLE 5
The `full` payload, written on the discrete GPU, resumed on other devices. Gaps are held-out loss against the same payload resumed on the discrete GPU, in mnats, over 200 steps. `fp32_nodrop`: single precision, no dropout. `as_trained`: bfloat16 autocast and dropout, as in the main sweep.

Model	Step	Device	Condition	Arrival	Mean	Max	ε	Max/ε
gpt-s	600	xpu	as_trained	0.0013	2.795	9.717	18.6	0.52
gpt-s	1700	xpu	as_trained	0.0168	1.698	3.591	10.0	0.36
gpt-s	600	cpu	fp32_nodrop	0.0	0.0	0.001	18.6	0.00
gpt-s	600	xpu	fp32_nodrop	0.0	0.0	0.0	18.6	0.00
gpt-s	1700	cpu	fp32_nodrop	0.0	0.0	0.0	10.0	0.00
gpt-s	1700	xpu	fp32_nodrop	0.0	0.0	0.0	10.0	0.00
gpt-xs	600	xpu	as_trained	0.0043	2.44	10.886	35.2	0.31
gpt-xs	1700	xpu	as_trained	0.0045	0.899	3.567	18.4	0.19
gpt-xs	600	cpu	fp32_nodrop	0.0	0.002	0.011	35.2	0.00
gpt-xs	600	xpu	fp32_nodrop	0.0	4.255	10.737	35.2	0.31
gpt-xs	1700	cpu	fp32_nodrop	0.0002	0.0	0.0	18.4	0.00
gpt-xs	1700	xpu	fp32_nodrop	0.0002	0.029	0.102	18.4	0.01

We ask instead whether the difference is larger than the difference between two seeds.

We wrote the full payload on the discrete GPU and resumed it on the CPU and on the integrated GPU of another vendor, then compared each continuation with the one on the discrete GPU (Table 5). In single precision without dropout, the CPU tracks the GPU to within a hundredth of a millinat over 200 steps. The integrated GPU is as close in three cells of four and drifts by up to 0.31 of the seed band in the fourth. Under the training configuration of the main sweep, with bfloat16 autocast and dropout, the integrated GPU stays within 0.52 of the band. Every cross-device gap we measured is inside the noise floor. A device change is, to the loss, a smaller event than a change of seed.

Three qualifications. The integrated GPU runs a newer release of the framework than the other two devices, so its rows change device and software together. We did not run the CPU under bfloat16 autocast, where it is far slower. And the laptop's neural processing unit does not appear, because it cannot train (Section 9).

6.6 End-to-end preemption

The measurements so far take a migration apart. Here we put it together and let it happen to a live job. A launcher runs the trainer as a separate process; we send it two preemption notices and kill it once without notice, at steps chosen in advance, and compare the result with the same job left alone (Fig. 8, Table 6). On a notice the job drains the rank-one payload and exits; the launcher starts a new process, which reads the payload and continues. On the kill, the new process falls back to the last periodic checkpoint, which is written every hundred steps with quantised moments.

The job finished its 2000 steps having computed one step twice, the one that was in flight when it was killed. The drain took at most a fifth of a second. Mean time to recovery was 5.257 s, and almost all of it is starting a process and initialising the GPU; fetching and decoding the payload took between a tenth and a fifth of a second. The whole run took 384 s of wall-clock time against 359 s for the control. The held-out loss is bit-identical to the control's up to the first preemption, wanders within a few millinats of it afterwards, well inside the seed band, and ends 1.8 mnats away.

We repeat the exercise in the supplement with two CPU ranks, where a preemption also shrinks the job from two ranks to one and a later one grows it back. Because batches are a function of the step and not of the rank count, the resized job sees the same data; it ends 13.7 mnats from its control.

7 A Migration Policy

A scheduler deciding what to send needs three numbers per strategy: its bytes, whether those bytes drain in time, and what the destination pays to recover. The first is exact for any architecture, since payload size depends only on tensor shapes and codecs (Table 2). The second follows from the bandwidth. The third is what we measured.

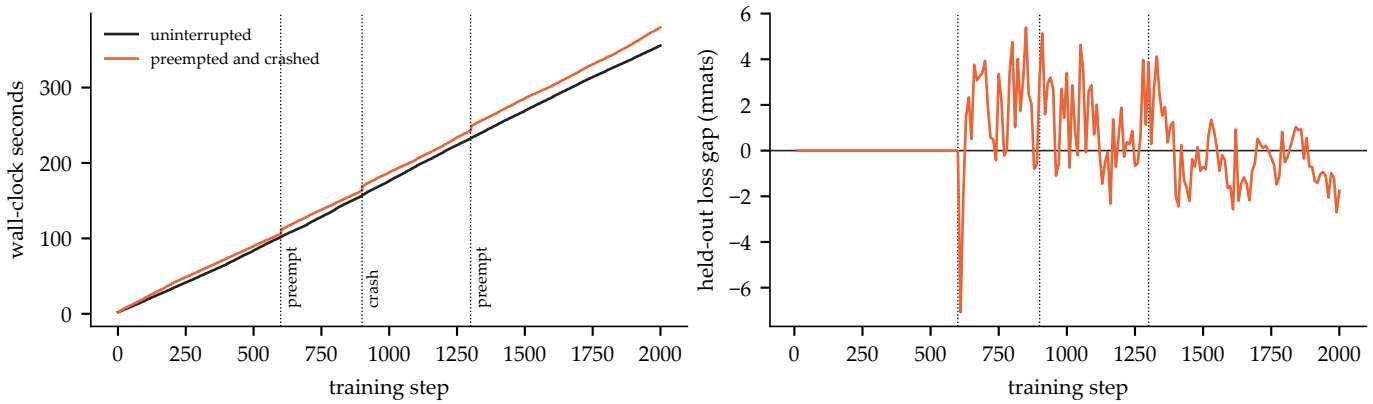


Fig. 8. One training run (10.7M parameters) hit by two preemptions and a crash, against the same run left alone. Left: wall-clock time against training step; each interruption shows as a small jump. Right: the difference in held-out loss between the two runs, which is exactly zero until the first lossy payload is used.

TABLE 6
Injected preemptions and crashes during one training run on the discrete GPU. A preemption drains `w_vr1` on notice; a crash falls back to the last periodic `w_mq8_vlog8` checkpoint. Times in seconds unless marked.

Event	Step	World	Steps redone	Payload (MB)	Drain	Restart	Init	Fetch (ms)	Rebuild (ms)	Downtime
preempt	600	1→1	0	43.166	0.096	2.047	1.683	109.78	13.18	5.274
crash	900	1→1	1	65.006	–	2.069	1.809	199.37	12.77	4.641
preempt	1300	1→1	0	43.166	0.207	2.143	1.958	133.54	22.03	5.857

Model. Let a strategy s have payload B_s bytes and cost λ_s steps of lost progress, and let the job have bandwidth w, notice window N, step time τ, G accelerators at price p per accelerator-hour, and egress price e per gigabyte. The drain takes $x_s = B_s/w$. If $x_s > N$ the payload does not leave in time; the job restarts from its last periodic checkpoint and loses, in expectation, half a checkpoint interval. Otherwise the migration costs

$$C_s = e\,B_s + G\,p\,(x_s + \lambda_s \tau), \tag{4}$$

bytes on the wire plus accelerators idle during the drain and busy re-earning lost progress. The policy picks the feasible strategy with the lowest C_s. A job that moves by choice, toward a cheaper region, moves only if the price difference over its remaining steps exceeds C_s.

An illustrative case. Table 7 applies the model to the shapes of a 6.74-billion parameter decoder [52] with $G = 8$, $p = \$1.20$, $\tau = 6\,\text{s}$, $e = \$0.02$ per gigabyte and a checkpoint every 1000 steps. *These rows are extrapolations.* The byte counts are exact for those shapes. The values of λ_s are those measured at 25.3M parameters for a migration at 60% of the schedule, held constant, and nothing here was trained at that size. With that caveat the structure of the answer is robust to the exact λ_s, because it is driven by feasibility. The full state, 80.9 GB, needs 64.7 s at 10 Gb/s. It fits a two-minute notice and misses a thirty-second one, where a job that insists on it is thrown back to its last checkpoint. The quantised state (40.7 GB, 32.6 s) still misses. The rank-one strategies (27.0 GB in 21.6 s, or 13.5 GB in 10.8 s with half-precision weights) fit with room to spare, and are the ones Section 6.1 found inside the noise floor. Fig. 9 shows the same boundary across bandwidths.

Reading the measurements as a rule. Combined with Sections 6.1–6.3, the policy reduces to an order of preference.

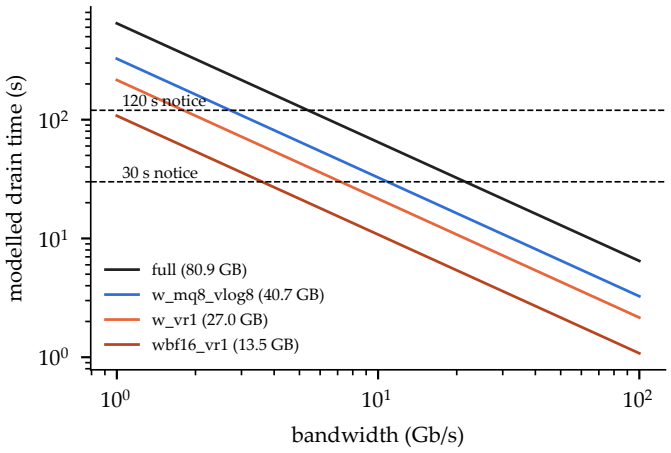


Fig. 9. Modelled drain time of four payloads for a 6.74-billion-parameter model against bandwidth (extrapolated from tensor shapes; not measured). A payload above a dashed line does not leave within that notice.

Send the quantised moments if they fit, since they cost nothing measurable. Otherwise drop m and send the rank-one sketch of v, with the weights in half precision if the window is still too short. If only the weights can leave, the destination should re-estimate the second moment with a few frozen passes and leave the first at zero, or, if it cannot spare the passes, build a new optimizer and ramp the learning rate again. It should not keep the step counter over empty moments, and it should not re-estimate the first moment. Late in training the choice hardly matters and the smallest payload is the right one; early in training it matters most (Section 6.2).

TABLE 7
Policy output for a 6.74-billion-parameter model. **Extrapolated**: payload sizes are computed from tensor shapes; recovery costs are those measured at 25.3M parameters. Drain times are bytes over bandwidth and exclude encoding.

			`full`			Chosen strategy			
Scenario	Notice (s)	Gb/s	Drain (s)	Fits	Strategies that fit	Name	GB	Drain (s)	Cost ($)
AWS, 10 Gb/s	120	10	64.7	yes	15/15	`wbf16_vr1`	13.5	10.8	0.41
AWS, 100 Gb/s	120	100	6.5	yes	15/15	`wbf16_vr1`	13.5	1.1	0.38
Azure, 10 Gb/s	30	10	64.7	no	9/15	`wbf16_vr1`	13.5	10.8	0.41
Azure, 100 Gb/s	30	100	6.5	yes	15/15	`wbf16_vr1`	13.5	1.1	0.38
GCP, 10 Gb/s	30	10	64.7	no	9/15	`wbf16_vr1`	13.5	10.8	0.41
GCP, 50 Gb/s	30	50	12.9	yes	15/15	`wbf16_vr1`	13.5	2.2	0.38

8 Discussion

Why the state comes apart. The three tensors remember different things. The weights hold the whole run. The second moment is a scale, averaged over $1/(1-\beta_2)$ steps, and it enters the update only through a square root, so a relative error in it becomes a relative error half as large in the step. The first moment is a direction, averaged over about ten steps. What it costs to leave a piece behind is set by two properties of that piece: how long it takes to refill, and whether the step is too short or too long while it does. An empty first moment refills in ten steps and shortens the step in the meantime. An empty second moment refills in a hundred and lengthens it. That asymmetry, and not the byte count, orders Table 3: `w_v` and `w_m` send the same bytes and their peaks differ by a factor of more than two hundred. It also explains the one strategy that pays for bytes it does not need. Carrying the first moment next to a sketched second one, `w_m_vr1`, lowers the peak from 0.31 to 0.17 of the band, and both are inside it, at twice the payload.

Relation to earlier findings. The conjecture of Section 1 holds at this scale: a run that could have lived with a coarse second moment at every step [14], [17] is not disturbed by being handed one once. Two results qualify what is known. First, the error of the first restored update, the acceptance test of Zhang et al. [20], is a safe screen but an incomplete one. `w_v` takes a first step half the true size with a cosine of 0.45 and would fail any tight bound on that error, yet it never leaves the band, because the error is a shrinkage. `w_zero` and `w_fresh` take the same first step, 5.0 and 5.2 times too long, and end at 11.40 and 4.38 bands, because what separates them happens over the following steps. The direction of the error and its lifetime matter as much as its size. Second, a weights-only restart is not one condition. Mitra [44] reports that it costs final quality and that rebuilding the first moment recovers little of the loss; here the five rules that send the weights alone span a factor of nearly thirty in peak, and the rule that rebuilds the first moment is among the worst, for the reason given in Section 4. Our window ends 300 steps after arrival, so we measure the transient and say nothing about quality at the end of a long run.

What a system can use. Three things follow without any change to how a job trains. The changeover delay that spot schedulers treat as a constant [1] can be replaced by a short list of pairs, bytes and recovery cost, one per strategy, from which the scheduler picks inside the notice it has been given. The same payloads serve as periodic checkpoints: the run of Section 6.6 wrote its checkpoints with quantised moments, at half the bytes of a full one. And the destination rule is free to change, since it costs no bytes. Loading weights into a newly built optimizer is `w_fresh`; 20 forward-backward passes before the first step turn it into `w_warmv` and take the peak from 4.38 bands to 0.40. The method is reusable in the same way. A bit-identical control and a seed band make an acceptance test for any lossy checkpoint, and both are cheap once data order is a function of the step.

Where we expect the picture to change. These are predictions of the model and were not tested. At $\beta_2 = 0.999$ the first update after a cold arrival is 13.8 times the steady one in place of 4.4 and an empty second moment takes ten times as long to refill, so dropping both moments should cost more and the sketch should be worth more. With a vocabulary of tens of thousands of tokens most embedding rows receive gradient rarely, their second moments are far from a product of row and column sums, and the strategy that keeps them exact may separate from the plain sketch, which it did not here. Early in training, at the peak learning rate, the margin of the rank-one strategies is already thin (Section 6.2). And the damage of dropping both moments grew with model size across our range, from 2.51 to 11.40 bands; we do not know whether it keeps growing.

9 Limitations

Scale. The largest model trained is 25.3M parameters, on one laptop GPU, on one small text dataset, with one setting of the decay rates. The mechanisms of Section 4 depend only on the decay rates and are not specific to a size, and the measured picture is stable across a factor of thirty in parameter count (Fig. 5), but none of this is evidence about a multi-billion-parameter run. Every figure quoted for such a model is an extrapolation and is labelled.

Three seeds. Each cell has 3 seeds, so each seed band is the 95th percentile over three pairs of controls and each interval resamples three units. The three groups of strategies are separated by more than an order of magnitude, which three seeds resolve; the differences inside a group, such as rank one against rank four, are not resolved and we do not claim them. The predicted dependence of a cold arrival on β_2 was not tested: every run uses $\beta_2 = 0.99$. The vocabulary is a few dozen characters, so the embedding results do not transfer to a tokenised model.

Simulated transport. A migration here serialises to local storage, tears the model and optimizer down and rebuilds them from the bundle. That reproduces exactly the information lost by a real migration, which is what the loss

measurements depend on. It does not reproduce a network: the drain times in Section 7 are bytes divided by an assumed bandwidth, and the measured latencies of Section 6.4 cover encoding and local I/O only. A two-machine migration over a real link has not been run.

One host. The multi-rank runtime was exercised with CPU ranks on one machine over Gloo. Resizing a job changes its world size, not its node count, and no claim is made about a GPU cluster.

Device classes. Three trainable devices were tested: an x86 CPU, an NVIDIA GPU and an Intel integrated GPU. The laptop also has a neural processing unit, and it was not used: current NPU software stacks execute inference graphs and offer no backward pass [53], [54], so a training job cannot be migrated to one. No TPU run was made. The integrated-GPU runs use a newer framework release than the others, so those rows change device and framework version together.

One optimizer family. All runs use AdamW. The analysis carries over to any method built on the same two exponential averages; it says nothing about second-order or sign-based methods, or about schedules with restarts.

Sharded state. Rank zero drains the whole payload. With tensor- or pipeline-parallel training the state is partitioned across devices and each shard would drain its own part; the strategies apply per tensor and do not depend on where a tensor lives, but that path was not built.

Steps lost. The cost model's λ_s comes from training loss and has a wide noise band. The peak, regret and recovery measures on held-out loss are the reliable ones, and our conclusions rest on those.

Concurrent work. The literature search was last run in November 2026. The two closest preprints are discussed in Section 2; others may have appeared since.

10 Conclusion

Systems for preemptible training have treated Adam's state as one indivisible thing and the cost of recovering from a move as a constant. We took the state apart and measured each piece against a bit-identical control and the spread between seeds. It comes apart cleanly. The first moment is a ten-step memory and can be left behind. The second moment is needed only as a scale, which its row and column sums supply in a third of a percent of its bytes. The second moment must not be dropped while the first is kept. And when nothing but the weights can cross, the outcome is decided at the destination: re-estimate the second moment and not the first, or reset the counter and warm the learning rate up again, but do not keep the counter over empty moments. A first moment cannot be rebuilt at the destination at all, because it records the path and not the place. Each of these statements is a prediction of a two-line model of the update, and each is borne out by the measurements.

For a scheduler the consequence is that the changeover delay is a choice. A third or a sixth of the bytes buys a transient that cannot be told from a change of seed, and in the cost model that turns a notice window the full state misses into one that is met with room to spare. Our measurements stop at 25.3M parameters, 3 seeds and one machine. The method does not: a deterministic control, a seed band and a censored recovery time are cheap to reproduce on any cluster. Whether the same curve holds at the sizes where the bytes hurt is one experiment for anyone who has the hardware, and we would very much like to know the answer.

Data and Code Availability

The implementation, every experiment configuration, the per-run manifests, the raw loss series and the scripts that regenerate each table, figure and in-text number of this paper from those series are available at https://github.com/riteshroshann/minstate. No number in the paper is typed by hand: the manuscript inputs a generated macro file.

Acknowledgment

The author thanks the School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, for its support. All experiments ran on the author's own laptop; no external compute grant was used.

References

[1] Z. Wu, W.-L. Chiang, Z. Mao, Z. Yang, E. Friedman, S. Shenker, and I. Stoica, "Can't be late: Optimizing spot instance savings under deadlines," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/wu-zhanghao

[2] S. Lee, J. Hwang, and K. Lee, "SpotLake: Diverse spot instance dataset archive service," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2022. [Online]. Available: https://arxiv.org/abs/2202.02973

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[5] S. Athlur, N. Saran, M. Sivathanu, R. Ramjee, and N. Kwatra, "Varuna: Scalable, low-cost training of massive deep learning models," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, 2022.

[6] J. Thorpe, P. Zhao, J. Eyolfson, Y. Qiao, Z. Jia, M. Zhang, R. Netravali, and G. H. Xu, "Bamboo: Making preemptible instances resilient for affordable training of large DNNs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/thorpe

[7] I. Jang, Z. Yang, Z. Zhang, X. Jin, and M. Chowdhury, "Oobleck: Resilient distributed training of large models using pipeline templates," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia, "Parcae: Proactive, liveput-optimized DNN training on preemptible instances," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/duan

[9] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. S. E. Ng, and Y. Wang, "GEMINI: Fast failure recovery in distributed training with in-memory checkpoints," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[10] A. Bhardwaj, W. Wang, J. Carin, A. Belay, and M. Ghobadi, "Checkmate: Zero-overhead model checkpointing via network gradient replication," 2025. [Online]. Available: https://arxiv.org/abs/2507.13522

[11] W. Liu, M. Li, R. Kang, C. Sun, G. Tan, and W. Jia, "Direct model state migration for elastic training of large language models," 2026. [Online]. Available: https://arxiv.org/abs/2607.04749

[12] R. Stoyanov, V. Spišaková, J. Ramos, S. Gurfinkel, A. Vagin, A. Reber, W. Armour, and R. Bruno, "CRIUgpu: Transparent checkpointing of GPU-accelerated workloads," 2025. [Online]. Available: https://arxiv.org/abs/2502.16631

[13] X. Wei, Z. Huang, T. Sun, Y. Hao, R. Chen, M. Han, J. Gu, and H. Chen, "PhoenixOS: Concurrent OS-level GPU checkpoint and restore with validated speculation," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, 2025.

[14] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," in *International Conference on Machine Learning (ICML)*, 2018. [Online]. Available: https://arxiv.org/abs/1804.04235

[15] Y. Zhang, C. Chen, Z. Li, T. Ding, C. Wu, D. P. Kingma, Y. Ye, Z.-Q. Luo, and R. Sun, "Adam-mini: Use fewer learning rates to gain more," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2406.16793

[16] D. S. Kalra, J. Kirchenbauer, M. Barkeshli, and T. Goldstein, "When can you get away with low memory Adam?" 2025. [Online]. Available: https://arxiv.org/abs/2503.01843

[17] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2110.02861

[18] B. Li, J. Chen, and J. Zhu, "Memory efficient optimizers with 4-bit states," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2309.01507

[19] W. Li, X. Chen, H. Shu, Y. Tang, and Y. Wang, "ExCP: Extreme LLM checkpoint compression via weight-momentum joint shrinking," in *International Conference on Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.11257

[20] A. Zhang, S. Song, W. Yao, L. Zhang, and Tianmiao, "Guarded compression of Adam optimizer state with shared-gradient bounds," 2026. [Online]. Available: https://arxiv.org/abs/2611.00589

[21] S. Gandhi, M. Zhao, A. Skiadopoulos, and C. Kozyrakis, "ReCycle: Resilient training of large DNNs using pipeline adaptation," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, 2024.

[22] Y. Wu, W. Qu, X. Liu, T. Tao, Y. Qiao, Z. Wang, W. Bai, Y. Tian, J. Zhang, Z. M. Mao, M. Lentz, D. Zhuo, and I. Stoica, "Lazarus: Resilient and elastic training of Mixture-of-Experts models," 2024. [Online]. Available: https://arxiv.org/abs/2407.04656

[23] C. Lao, J. Gao, J. Cao, Z. Zhang, P. Zhang, J. Duan, Z. Zheng, Y. Guan, Y. Xu, Y. Li, Z. Qian, A. Akella, M. Yu, E. Zhai, D. Cai, and J. Zhou, "TrainMover: An interruption-resilient runtime for ML training," 2024. [Online]. Available: https://arxiv.org/abs/2412.12636

[24] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing, "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021. [Online]. Available: https://www.usenix.org/conference/osdi21/presentation/qiao

[25] M. Li, W. Xiao, H. Yang, B. Sun, H. Zhao, S. Ren, Z. Luan, X. Jia, Y. Liu, Y. Li, W. Lin, and D. Qian, "EasyScale: Elastic training with consistent accuracy and improved utilization on GPUs," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.

[26] D. Shukla, M. Sivathanu, S. Viswanatha, B. Gulavani, R. Nehme, A. Agrawal, C. Chen, N. Kwatra, R. Ramjee, P. Sharma, A. Katiyar, V. Modi, V. Sharma, A. Singh, S. Singhal, K. Welankar, L. Xun, R. Anupindi, K. Elangovan, H. Rahman, Z. Lin, R. Seetharaman, C. Xu, E. Ailijiang, S. Krishnappa, and M. Russinovich, "Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads," 2022. [Online]. Available: https://arxiv.org/abs/2202.07848

[27] Z. Yang, Z. Wu, M. Luo, W.-L. Chiang, R. Bhardwaj, W. Kwon, S. Zhuang, F. S. Luan, G. Mittal, S. Shenker, and I. Stoica, "SkyPilot: An intercloud broker for sky computing," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/yang-zongheng

[28] Z. Li, T. Xia, Z. Mao, Z. Zhou, E. J. Jackson, J. Kerney, Z. Wu, P. Mishra, Y. Xu, Y. Qiao, S. Shenker, and I. Stoica, "SkyNomad: On using multi-region spot instances to minimize AI batch job cost," 2026. [Online]. Available: https://arxiv.org/abs/2601.06520

[29] J. Mohan, A. Phanishayee, and V. Chidambaram, "CheckFreq: Frequent, fine-grained DNN checkpointing," in *19th USENIX Conference on File and Storage Technologies (FAST)*, 2021. [Online]. Available: https://www.usenix.org/conference/fast21/presentation/mohan

[30] T. Gupta, S. Krishnan, R. Kumar, A. Vijeev, B. Gulavani, N. Kwatra, R. Ramjee, and M. Sivathanu, "Just-In-Time checkpointing: Low cost error recovery from deep learning training failures," in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, 2024.

[31] Y. Zhong, G. Sheng, J. Liu, J. Yuan, and C. Wu, "Swift: Expedited failure recovery for large-scale DNN training," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 9, 2024.

[32] A. Maurya, M. M. Rafique, F. Cappello, and B. Nicolae, "DataStates-LLM: Scalable checkpointing for transformer models using composable state providers," *IEEE Transactions on Parallel and Distributed Systems*, 2026.

[33] A. Eisenman, K. K. Matam, S. Ingram, D. Mudigere, R. Krishnamoorthi, K. Nair, M. Smelyanskiy, and M. Annavaram, "Check-N-Run: A checkpointing system for training deep learning recommendation models," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2022. [Online]. Available: https://www.usenix.org/conference/nsdi22/presentation/eisenman

[34] A. Agrawal, S. Reddy, S. Bhattamishra, V. P. S. Nookala, V. Vashishth, K. Rong, and A. Tumanov, "Inshrinkerator: Compressing deep learning training checkpoints via dynamic quantization," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2024.

[35] W. Cai, L. Qin, and J. Huang, "MoC-System: Efficient fault tolerance for sparse Mixture-of-Experts model training," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[36] S. Gandhi and C. Kozyrakis, "Sparse checkpointing for fast and reliable MoE training," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/gandhi

[37] A. Kokolis, M. Kuchnik, J. Hoffman, A. Kumar, P. Malani, F. Ma, Z. DeVito, S. Sengupta, K. Saladi, and C.-J. Wu, "Revisiting reliability in large-scale machine learning research clusters," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025.

[38] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the variance of the adaptive learning rate and beyond," in *International Conference on Learning Representations (ICLR)*, 2020. [Online]. Available: https://arxiv.org/abs/1908.03265

[39] J. Ma and D. Yarats, "On the adequacy of untuned warmup for adaptive optimization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021. [Online]. Available: https://arxiv.org/abs/1910.04209

[40] D. S. Kalra and M. Barkeshli, "Why warmup the learning rate? Underlying mechanisms and improvements," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09405

[41] S. Laing and A. Orvieto, "Adam simplified: Bias correction debunked," 2025. [Online]. Available: https://arxiv.org/abs/2511.20516

[42] J. M. Cohen, B. Ghorbani, S. Krishnan, N. Agarwal, S. Medapati, M. Badura, D. Suo, D. Cardoze, Z. Nado, G. E. Dahl, and J. Gilmer, "Adaptive gradient methods at the edge of stability," 2022. [Online]. Available: https://arxiv.org/abs/2207.14484

[43] K. Asadi, R. Fakoor, and S. Sabach, "Resetting the optimizer in deep RL: An empirical study," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[44] S. Mitra, "When is availability-aware training worth it? A benchmark and empirical study of interruption-resilient optimization under predictable compute schedules," 2026. [Online]. Available: https://arxiv.org/abs/2609.22087

[45] C. Summers and M. J. Dinneen, "Nondeterminism and instability in neural network optimization," in *International Conference on Machine Learning (ICML)*, 2021. [Online]. Available: https://arxiv.org/abs/2103.04514

[46] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou,

V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021. [Online]. Available: https://arxiv.org/abs/2103.03098

[47] O. van der Wal, P. Lesci, M. Müller-Eberstein, N. Saphra, H. Schoelkopf, W. Zuidema, and S. Biderman, "PolyPythias: Stability and outliers across fifty language model pre-training runs," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2503.09543

[48] PyTorch Contributors, "Reproducibility," PyTorch 2.15 documentation, 2026. [Online]. Available: https://docs.pytorch.org/docs/2.15/notes/randomness.html

[49] H. V. Pham, S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan, "Problems and opportunities in training deep learning software systems: An analysis of variance," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020.

[50] D. Zhuang, X. Zhang, S. L. Song, and S. Hooker, "Randomness in neural network training: Characterizing the impact of tooling," 2021. [Online]. Available: https://arxiv.org/abs/2106.11872

[51] A. Karpathy, "nanoGPT," GitHub repository, 2022. [Online]. Available: https://github.com/karpathy/nanoGPT

[52] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[53] Intel Corporation, "NPU device," OpenVINO documentation, 2025. [Online]. Available: https://docs.openvino.ai/2025/openvino-workflow/running-inference/inference-devices-and-modes/npu-device.html

[54] C. Feng, S. Zhuo, X. Zhang, R. K. Ramakrishnan, Z. Yuan, and A. Z. Li, "Stepping forward on the last mile," 2024. [Online]. Available: https://arxiv.org/abs/2411.04036

Ritesh Roshan Sahoo is with the School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Faridabad, India. His research interests are distributed training systems, optimization for deep learning and high-performance computing.